Assignment: Smart Task Management System

Name:Sayali Hanmant Koshti

Gmail: Koshtisayali5@gmail.com

Githup: https://github.com/Skoshti987/smart-task-manager

Database File/Schema

```
-- Smart Task Manager Database Schema
-- PostgreSQL Database Schema

-- Create database (uncomment if needed)
-- CREATE DATABASE smart_task_db;
-- CREATE DATABASE smart_task_db_dev;

-- Use the database
-- \c smart_task_db;

-- Drop existing tables if they exist (for development)
DROP TABLE IF EXISTS tasks CASCADE;
DROP TABLE IF EXISTS users CASCADE;

-- Create Users table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
```

```
    username VARCHAR(80) UNIQUE NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create Tasks table
CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    description TEXT,
    priority VARCHAR(20) DEFAULT 'medium' CHECK (priority IN ('low', 'medium', 'high')),
    status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'in_progress', 'completed')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    due_date TIMESTAMP,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE
);

-- Create indexes for better performance
CREATE INDEX idx_tasks_user_id ON tasks(user_id);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_priority ON tasks(priority);
CREATE INDEX idx_tasks_created_at ON tasks(created_at);
CREATE INDEX idx_tasks_due_date ON tasks(due_date);
```

```
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email ON users(email);

-- Create trigger to automatically update updated_at timestamp
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_tasks_updated_at
    BEFORE UPDATE ON tasks
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();

-- Insert sample data for testing (optional)
INSERT INTO users (username, email, password_hash) VALUES
('admin', 'admin@example.com',
'$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewdBPj6ukx.LFvOe'), --
password: admin123
('john_doe', 'john@example.com',
'$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewdBPj6ukx.LFvOe'), --
password: admin123
('jane_smith', 'jane@example.com',
'$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewdBPj6ukx.LFvOe'); -
- password: admin123
```

```
-- Insert sample tasks for testing (optional)

INSERT INTO tasks (title, description, priority, status, due_date, user_id)
VALUES

('Complete project documentation', 'Write comprehensive documentation for the Smart Task Manager project', 'high', 'in_progress', CURRENT_TIMESTAMP + INTERVAL '3 days', 1),

('Review pull requests', 'Review and merge pending pull requests from team members', 'medium', 'pending', CURRENT_TIMESTAMP + INTERVAL '1 day', 1),

('Update dependencies', 'Update all npm and pip dependencies to latest stable versions', 'low', 'completed', CURRENT_TIMESTAMP - INTERVAL '2 days', 1),

('Fix login bug', 'Investigate and fix the login authentication issue reported by users', 'high', 'pending', CURRENT_TIMESTAMP + INTERVAL '1 day', 2),

('Prepare presentation', 'Create slides for the upcoming team meeting presentation', 'medium', 'pending', CURRENT_TIMESTAMP + INTERVAL '5 days', 2),

('Code review', 'Review the new analytics module implementation', 'medium', 'completed', CURRENT_TIMESTAMP - INTERVAL '1 day', 3),

('Database optimization', 'Optimize database queries for better performance', 'high', 'in_progress', CURRENT_TIMESTAMP + INTERVAL '2 days', 3),

('User testing', 'Conduct user testing sessions for the new features', 'low', 'pending', CURRENT_TIMESTAMP + INTERVAL '7 days', 3);


-- Create views for common queries

CREATE VIEW task_summary AS

SELECT

    u.id as user_id,

    u.username,

    COUNT(t.id) as total_tasks,

    COUNT(CASE WHEN t.status = 'completed' THEN 1 END) as completed_tasks,
```

```
    COUNT(CASE WHEN t.status = 'pending' THEN 1 END) as pending_tasks,
    COUNT(CASE WHEN t.status = 'in_progress' THEN 1 END) as
in_progress_tasks,
    COUNT(CASE WHEN t.due_date < CURRENT_TIMESTAMP AND t.status !=
'completed' THEN 1 END) as overdue_tasks,
    ROUND(
        (COUNT(CASE WHEN t.status = 'completed' THEN 1 END) * 100.0 /
NULLIF(COUNT(t.id), 0)), 2
    ) as completion_percentage
FROM users u
LEFT JOIN tasks t ON u.id = t.user_id
GROUP BY u.id, u.username;

CREATE VIEW priority_analysis AS
SELECT
    u.id as user_id,
    u.username,
    t.priority,
    COUNT(t.id) as total_by_priority,
    COUNT(CASE WHEN t.status = 'completed' THEN 1 END) as
completed_by_priority,
    COUNT(CASE WHEN t.status = 'pending' THEN 1 END) as
pending_by_priority,
    COUNT(CASE WHEN t.status = 'in_progress' THEN 1 END) as
in_progress_by_priority
FROM users u
LEFT JOIN tasks t ON u.id = t.user_id
GROUP BY u.id, u.username, t.priority
```

```
ORDER BY u.id, t.priority;


-- Create function to get productivity score
CREATE OR REPLACE FUNCTION calculate_productivity_score(p_user_id
INTEGER)
RETURNS DECIMAL(5,2) AS $$
DECLARE
    v_total_tasks INTEGER;
    v_completed_tasks INTEGER;
    v_high_priority_tasks INTEGER;
    v_high_priority_completed INTEGER;
    v_overdue_tasks INTEGER;
    v_completion_score DECIMAL(5,2);
    v_high_priority_bonus DECIMAL(5,2);
    v_overdue_penalty DECIMAL(5,2);
    v_final_score DECIMAL(5,2);
BEGIN
    -- Get basic metrics
    SELECT COUNT(t.id), COUNT(CASE WHEN t.status = 'completed' THEN 1 END)
    INTO v_total_tasks, v_completed_tasks
    FROM tasks t
    WHERE t.user_id = p_user_id;

    -- Get high priority metrics
    SELECT COUNT(t.id), COUNT(CASE WHEN t.status = 'completed' THEN 1 END)
    INTO v_high_priority_tasks, v_high_priority_completed
    FROM tasks t
```

```
    WHERE t.user_id = p_user_id AND t.priority = 'high';

    -- Get overdue tasks count
    SELECT COUNT(t.id)
    INTO v_overdue_tasks
    FROM tasks t
    WHERE t.user_id = p_user_id
    AND t.status != 'completed'
    AND t.due_date < CURRENT_TIMESTAMP;

    -- Calculate base completion score
    IF v_total_tasks > 0 THEN
        v_completion_score := (v_completed_tasks * 100.0 / v_total_tasks);
    ELSE
        v_completion_score := 0;
    END IF;

    -- Calculate high priority bonus (max 20 points)
    IF v_high_priority_tasks > 0 THEN
        v_high_priority_bonus := LEAST((v_high_priority_completed * 100.0 /
v_high_priority_tasks), 20);
    ELSE
        v_high_priority_bonus := 0;
    END IF;

    -- Calculate overdue penalty (5 points per overdue task, max 30 points)
    v_overdue_penalty := LEAST(v_overdue_tasks * 5, 30);
```

```
    -- Calculate final score
    v_final_score := v_completion_score + v_high_priority_bonus -
v_overdue_penalty;
    v_final_score := GREATEST(0, LEAST(100, v_final_score));

    RETURN ROUND(v_final_score, 2);
END;
$$ LANGUAGE plpgsql;

-- Grant permissions (adjust as needed for your setup)
-- GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO
your_username;
-- GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO
your_username;

-- Create indexes for views
CREATE INDEX idx_task_summary_user_id ON task_summary(user_id);
CREATE INDEX idx_priority_analysis_user_id ON priority_analysis(user_id);

-- Comments for documentation
COMMENT ON TABLE users IS 'User accounts for the Smart Task Manager
application';
COMMENT ON TABLE tasks IS 'Tasks created and managed by users';
COMMENT ON COLUMN tasks.priority IS 'Task priority level: low, medium, or
high';
COMMENT ON COLUMN tasks.status IS 'Task status: pending, in_progress, or
completed';
```

```
COMMENT ON COLUMN tasks.due_date IS 'Optional due date for task completion';

COMMENT ON VIEW task_summary IS 'Summary view showing task statistics per user';

COMMENT ON VIEW priority_analysis IS 'Analysis view showing task breakdown by priority per user';

COMMENT ON FUNCTION calculate_productivity_score IS 'Calculates productivity score based on completion rate, priority handling, and overdue tasks';


-- Sample queries for testing

/*

-- Get all users and their task summary

SELECT * FROM task_summary;


-- Get priority analysis for a specific user

SELECT * FROM priority_analysis WHERE user_id = 1;


-- Calculate productivity score for a user

SELECT calculate_productivity_score(1) as productivity_score;


-- Get overdue tasks

SELECT t.*, u.username

FROM tasks t

JOIN users u ON t.user_id = u.id

WHERE t.due_date < CURRENT_TIMESTAMP AND t.status != 'completed';


-- Get tasks due in the next 7 days
```

SELECT t.*, u.username

FROM tasks t

JOIN users u ON t.user_id = u.id

WHERE t.due_date BETWEEN CURRENT_TIMESTAMP AND
CURRENT_TIMESTAMP + INTERVAL '7 days'

AND t.status != 'completed'

ORDER BY t.due_date;

*/

README.md with setup steps

 # Smart Task Management System

A comprehensive full-stack web application for task management with real-time updates, analytics, and PostgreSQL database integration.

Features

Core Functionality

- ****User Authentication****: Registration, login, and secure session management

- ****Task Management****: Create, read, update, and delete tasks

- ****Real-time Updates****: WebSocket integration for live task updates

- ****Analytics Dashboard****: Comprehensive task analytics with charts and productivity scores

- ****Priority Management****: High, medium, and low priority task classification

- ****Due Date Tracking****: Task scheduling and overdue task notifications

Technical Features

- **Backend**: Flask with SQLAlchemy ORM
- **Database**: PostgreSQL with optimized queries and indexes
- **Frontend**: Bootstrap 5 with responsive design
- **Real-time Communication**: Socket.IO for WebSocket connections
- **Data Analytics**: Pandas and NumPy for statistical analysis
- **Security**: Password hashing, input validation, and CSRF protection

Technology Stack

Backend

- **Flask 2.3.3** - Web framework
- **Flask-SQLAlchemy 3.0.5** - ORM
- **Flask-Login 0.6.3** - Authentication
- **Flask-SocketIO 5.3.6** - WebSocket support
- **psycopg2-binary** - PostgreSQL adapter
- **pandas 2.0.3** - Data analysis
- **numpy 1.24.3** - Numerical computing

Frontend

- **Bootstrap 5.3.0** - UI framework
- **Font Awesome 6.4.0** - Icons
- **Chart.js** - Data visualization
- **Socket.IO Client** - Real-time communication

Database

- **PostgreSQL** - Primary database
- **SQLite** - Development database
- **SQLAlchemy ORM** - Database abstraction

Project Structure

```
smart-task-manager/
|
├── app.py              # Main Flask application
├── config_dev.py       # Development configuration
├── requirements.txt    # Python dependencies
├── README.md           # Project documentation
├── extensions.py       # Flask extensions
├── run_dev.py          # Development server
|
├── models/             # Database models
|   └── __init__.py     # User and Task models
|
├── routes/             # API routes
|   ├── auth_routes.py  # Authentication endpoints
|   ├── task_routes.py  # Task management endpoints
|   └── analytics_routes.py # Analytics endpoints
|
├── templates/          # HTML templates
|   ├── base.html       # Base template
```

```
|   ├── login.html       # Login page
|   ├── register.html    # Registration page
|   └── dashboard.html   # Main dashboard
|
├── static/             # Static assets
|   ├── css/
|   |   └── style.css   # Custom styles
|   ├── js/
|   |   └── script.js   # JavaScript functionality
|   └── images/         # Image assets
|
├── utils/              # Utility functions
|   └── analytics.py     # Analytics calculations
|
└── database/           # Database files
    └── schema.sql      # Database schema
```

Installation and Setup

Prerequisites

- Python 3.8+
- PostgreSQL 12+ (for production)
- Git (for cloning)

1. Clone the Repository

```bash
git clone https://github.com/yourusername/smart-task-manager.git
cd smart-task-manager
```

2. Set Up Virtual Environment

```bash
python -m venv venv
# On Windows
venv\Scripts\activate
# On macOS/Linux
source venv/bin/activate
```

3. Install Dependencies

```bash
pip install -r requirements.txt
```

4. Run the Application

Development Mode (Recommended)

```bash
python run_dev.py
```

Production Mode

```bash

# Set environment variables

export FLASK_CONFIG=production

export DATABASE_URL=postgresql://username:password@localhost/smart_task_db

export SECRET_KEY=your-production-secret-key

# Run with production server

gunicorn --worker-class eventlet -w 1 app:app
```

5. Access the Application

- **URL**: http://localhost:5000

- **Test Credentials**: admin/admin123

API Endpoints

Authentication

- `POST /auth/register` - Register new user

- `POST /auth/login` - User login

- `GET /auth/logout` - User logout

- `GET /auth/current-user` - Get current user info

Tasks

- `POST /tasks/add-task` - Create new task

- `GET /tasks/tasks` - Get all tasks (with filters)

- `PUT /tasks/update-task/<id>` - Update task
- `DELETE /tasks/delete-task/<id>` - Delete task
- `GET /tasks/task/<id>` - Get specific task
- `POST /tasks/bulk-update` - Bulk update tasks

Analytics

- `GET /analytics` - Get all analytics
- `GET /analytics/basic` - Get basic analytics
- `GET /analytics/priority` - Get priority analytics
- `GET /analytics/time` - Get time-based analytics
- `GET /analytics/productivity-score` - Get productivity score
- `GET /analytics/dashboard` - Get dashboard analytics

WebSocket Events

Client to Server

- `connect` - Establish connection
- `join_user_room` - Join user-specific room
- `leave_user_room` - Leave user-specific room

Server to Client

- `task_added` - New task created
- `task_updated` - Task modified
- `task_deleted` - Task removed
- `tasks_bulk_updated` - Multiple tasks updated

Database Schema

Users Table

- `id` (Primary Key)
- `username` (Unique)
- `email` (Unique)
- `password_hash`
- `created_at`

Tasks Table

- `id` (Primary Key)
- `title`
- `description`
- `priority` (low, medium, high)
- `status` (pending, in_progress, completed)
- `created_at`
- `updated_at`
- `due_date`
- `user_id` (Foreign Key)

Analytics Features

Basic Analytics

- Total tasks count
- Completed tasks count
- Pending tasks count

- In-progress tasks count
- Completion percentage
- Overdue tasks count

Priority Analytics
- Task distribution by priority
- Completion rates by priority level
- Priority-based performance metrics

Time Analytics
- Tasks created in time period
- Daily task creation averages
- Completion trends over time
- Productivity patterns

Productivity Score
- Calculated based on completion rate
- Bonus for high-priority task completion
- Penalty for overdue tasks
- Score range: 0-100

Security Features

- Password hashing with Werkzeug
- Input validation and sanitization
- CSRF protection

- Secure session management
- SQL injection prevention through ORM
- XSS protection with template escaping

Performance Optimizations

- Database indexes on frequently queried columns
- Optimized database queries
- Efficient WebSocket room management
- Client-side caching for static assets
- Lazy loading for large datasets

GitHub Setup and Upload

1. Initialize Git Repository

```bash
git init
git add .
git commit -m "Initial commit: Smart Task Manager"
```

2. Create GitHub Repository

- Go to https://github.com and create new repository
- Name: `smart-task-manager`
- Choose Public/Private as needed
- Don't initialize with README (we already have one)

3. Push to GitHub

```bash
git remote add origin https://github.com/yourusername/smart-task-manager.git
git branch -M main
git push -u origin main
```

Database Setup (PostgreSQL)

Development (SQLite)

- Automatic setup with `python run_dev.py`
- Database file: `smart_task.db`

Production (PostgreSQL)

```bash
# Create database
createdb smart_task_db

# Import schema
psql -d smart_task_db -f database/schema.sql

# Set environment variable
export DATABASE_URL=postgresql://username:password@localhost/smart_task_db
```

Docker Deployment

```dockerfile
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["gunicorn", "--worker-class", "eventlet", "-w", "1", "app:app"]
```

Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests for new functionality
5. Run tests and ensure they pass
6. Submit a pull request

License

Support

For support and questions:

- Create an issue in the repository
- Check the documentation
- Review existing issues

Future Enhancements

- [ ] Mobile application
- [ ] Email notifications
- [ ] Task templates
- [ ] Team collaboration features
- [ ] Advanced reporting
- [ ] Integration with calendar applications
- [ ] File attachments for tasks
- [ ] Task dependencies
- [ ] Time tracking
- [ ] Kanban board view

Quick Start Guide

```bash
# Clone and setup
git clone https://github.com/yourusername/smart-task-manager.git
cd smart-task-manager
```

pip install -r requirements.txt

Run development server

python run_dev.py

Access at http://localhost:5000

Login: admin/admin123

```

## Project Highlights

This project demonstrates:

✅ **Full Stack Development** - Complete frontend and backend integration

✅ **Real-time Systems** - WebSocket implementation for live updates

✅ **Database Design** - PostgreSQL with optimized schema

✅ **API Architecture** - RESTful API design patterns

✅ **Python Data Analysis** - Pandas and NumPy integration

✅ **Modern Frontend** - Bootstrap 5 with responsive design

✅ **Security Best Practices** - Authentication and data protection

✅ **Production Ready** - Scalable architecture and deployment options

Perfect for:

- Internship portfolio projects

- Full-stack development demonstrations

- Database integration examples
```

- Real-time application showcases

- Academic assignments

4 Demo Video

https://drive.google.com/drive/folders/10iB5pcJwia_-aOwUpout9gSGHMguVxI2?usp=sharing